

Efficient Real-Time Fire Surveillance: A Lightweight Motion-Heuristic Skip Module for Accelerated Inference in Indoor Environments

Hoang Van-Ha, Jong Weon Lee, Park Chun-Su

2026.06.02

Abstract

Conventional deep learning (DL)-based fire and smoke detection systems perform inference on every video frame using high-complexity models to classify the presence of fire or smoke. However, in surveillance scenarios, particularly indoor environments, video streams frequently contain long periods of static background frames with little or no motion, rendering per-frame inference redundant and wasteful of computational resources. To address this limitation, this study proposes an efficient inference acceleration framework for fire and smoke surveillance systems that combines a motion-aware skip module with an Eager mode for recall recovery. The skip module leverages motion detection to selectively bypass unnecessary DL inference on non-informative frames, while the Eager mode periodically forces inference updates to mitigate missed detections from slow-developing or nearly stationary smoke events. Evaluation on a large-scale dataset comprising 150 indoor videos demonstrates that the proposed framework achieves nearly 30% higher throughput than the baseline performing per-frame inference, with only a 0.05% reduction in recall. In contrast, using the skip module alone results in a 1.23% recall drop, highlighting the effectiveness of the Eager mode in recovering detection performance while preserving most of the throughput gain. These results demonstrate that the proposed framework provides an effective plug-and-play inference acceleration strategy for real-time indoor fire and smoke surveillance systems.

1 Introduction

Fires and wildfires, if not detected and controlled in their early stages, can quickly escalate, especially under dry and windy conditions, resulting in catastrophic consequences, including loss of life, destruction of property, and damage to natural ecosystems. For instance, in 2017, urban fires and wildfires in California, USA, caused an estimated economic loss of 10 billion USD [1]. More recently, in 2025, a forest fire in Gyeongsang Province, South Korea, burned approximately 90,000 acres, resulting in at least 27 fatalities and the evacuation of nearly 40,000 residents [2]. These incidents highlight the critical importance of automated early-stage fire and smoke detection systems for enabling timely intervention and minimizing damage.

In recent years, DL-based approaches have emerged as the prevailing methodology in fire and smoke detection systems [3], [4]. In practical deployments, such systems are commonly integrated into CCTV video pipelines, where DL-based classifiers or object detectors perform inference on every individual frame. While this per-frame paradigm is straightforward to implement, it exhibits two major limitations. First, it relies solely on spatial information within isolated frames, neglecting temporal cues like motion video sequences. Second, in surveillance scenarios, particularly in **indoor** environments, video streams can exhibit minimal visual variation across consecutive frames due to static scene content. Indiscriminately applying computationally intensive DL models to every frame under these conditions introduces significant processing redundancy, increasing latency and computational overhead without improving detection performance. This inefficiency is further amplified by the well-known accuracy-efficiency trade-off in DL: high-accuracy models typically demand substantial computational resources, and indiscriminate frame processing amplifies resource demands, and indiscriminate per-frame inference magnifies these demands, posing a significant barrier to real-time deployment.

A common strategy for reducing inference cost is to substitute a high-complexity model with a lightweight alternative; however, this approach typically compromises detection reliability — an unacceptable trade-off in safety-critical applications such as fire and smoke surveillance. This study adopts a complementary approach: rather than replacing the classifier, we propose an efficient inference acceleration framework that introduces a lightweight, motion-aware skip module acting as a computational gate upstream of the classifier. The skip module leverages frame differencing-based motion estimation to selectively bypass DL inference on static, non-informative frames, while forwarding only frames with significant scene activity to the high-capacity classifier. To address a key failure mode of motion-based skipping — namely, missed detections arising from slow-developing or nearly stationary smoke events — the framework additionally incorporates an Eager mode that periodically forces inference updates to recover detection recall. As illustrated in Fig. 1, the conventional pipeline processes every frame through the classifier, whereas the proposed pipeline adds the skip module early in the process to conditionally suppress redundant inference calls.

In particular, our main contributions are summarized as follows:

- **Motion-Aware Skip Module:** A lightweight skip module is proposed that exploits frame differencing-based motion estimation to identify and bypass DL inference on static frames, thereby reducing computational cost without modifying the underlying classifier. The proposed module is designed as a plug-and-play component compatible with existing fire and smoke detection pipelines. A systematic hyperparameter optimization procedure is further developed to identify the optimal operating configuration for maximizing throughput while preserving detection performance.

- **Indoor Fire and Smoke Video Dataset:** We construct an annotated dataset comprising 150 indoor fire and smoke videos (including fire, smoke, and background-only classes) captured by static surveillance cameras at resolutions ranging from 814×720 to 3840×2160 . The dataset covers diverse environments such as warehouses, parking areas, and offices under varying lighting conditions, providing a realistic benchmark for evaluating detection systems in static surveillance scenarios.
- **Comprehensive System Evaluation and Analysis:** The proposed framework is integrated with a high-capacity DL classifier (BIG Model) and evaluated on the constructed dataset against a per-frame inference baseline and existing methods. Results demonstrate approximately 30% higher throughput than the baseline, with a recall reduction of roughly 0.05% when the Eager mode is active — compared to a 1.23% recall drop when the skip module is used alone — highlighting the effectiveness of the Eager mode in recovering detection performance while preserving most throughput gains. Beyond quantitative benchmarking, a detailed efficiency analysis of the proposed system is conducted, and success and failure cases.

The rest of this paper is organized as follows: Section 2 reviews existing fire and smoke detection methods for video and relevant motion detection techniques, contextualizing the need for efficient processing in static surveillance scenarios. Section 3 describes the proposed system architecture, the design of the skip-module, its integration with the classifier, the hyperparameter optimization strategy, and the Eager mode proposed to address missed detections arising from slow-developing or nearly stationary smoke events. Section 4 presents the experimental setup, evaluation metrics, and results on the large-scale indoor video dataset, including comparative analyses against the baseline and an existing temporally-aware method, and qualitative examination of success and failure cases of the proposed skip module. Finally, Section 5 summarizes the key findings and discusses limitations and future research directions.

2 Related Work

Fire and Smoke Detection in Images and Videos: Automated fire and smoke detection has attracted considerable research interest, with deep learning emerging as the dominant paradigm. The work [3] provides a comprehensive survey of visual fire detection methods, highlighting the rapid adoption of DL-based approaches in this domain. The majority of these methods formulate the problem as image-level binary classification (fire/smoke vs. none) or object detection, in which a model receives a single RGB image and outputs either a class label with confidence score or localized bounding boxes [5], [6]. Research efforts have primarily focused on improving model accuracy and inference speed through architectural modifications and advanced training strategies. In video-based deployments, these image classifiers are applied directly to each frame, enabling integration with existing pipelines without architectural changes.

However, processing frames independently discards temporal information inherent in video data that is potentially informative for detection. [7] survey video-based fire and smoke detection approaches and highlight the benefit of modeling temporal dynamics. Representative methods include the work of [8], who combine a CNN for spatial feature extraction with a bidirectional LSTM for temporal modeling, and [9], who employ 3D CNNs with attention mechanisms to jointly capture spatio-temporal patterns. While these approaches demonstrate improved detection performance over purely image-based methods, they introduce greater computational cost and require more laborious dataset construction involving temporal annotation. Hybrid methods address this partially by combining image-based classifiers with lightweight temporal post-processing — such as temporal voting [10] or motion-based false positive suppression [11] — yet still apply DL inference unconditionally to every frame, regardless of scene content.

Across both paradigms, the dominant deployment pattern applies DL inference to each frame or short clip, without considering whether inference is warranted given the frame content. In surveillance scenarios, particularly **indoors**, video streams frequently consist of purely background frames with no motion, in which fire or smoke is generally unlikely. Motivated by this observation, we propose a skip module that filters such motion-free frames prior to DL inference, reducing computational cost while with minimal degradation in detection reliability. The module prioritizes retaining positive frames (fire/smoke present) while skipping negative frames (background only), functioning as a complementary accelerator for any existing frame-wise detection pipeline.

Motion Detection and the Proposed Skip Module: The skip decision in our module relies on motion estimation, which connects to the well-studied problem of background subtraction. [12] and [13] provide comprehensive reviews of background subtraction methods, ranging from simple statistical models to deep learning-based approaches. More robust methods such as MOG2 and KNN [14] model the scene background statistically and are resilient to gradual illumination changes; however, they introduce non-trivial memory overhead and initialization latency. As the skip module is designed primarily as a fast, lightweight gate, we instead adopt motion estimation based on frame differencing, which offers minimal computational overhead and requires no background model initialization.

Existing efficient video inference methods, such as FrameExit [15], employ learned gating mechanisms to selectively process frames based on prediction confidence, thereby reducing redundant inference; however, they require joint training of the gating mechanism and the underlying classifier, limiting plug-and-play applicability to pre-trained models. In contrast, this study adopts a training-free approach by integrating lightweight frame-differencing-based motion estimation as a plug-and-play gate, requiring no retraining of the underlying classifier, as described in Section 3.

3 The proposed method

3.1 System Overview

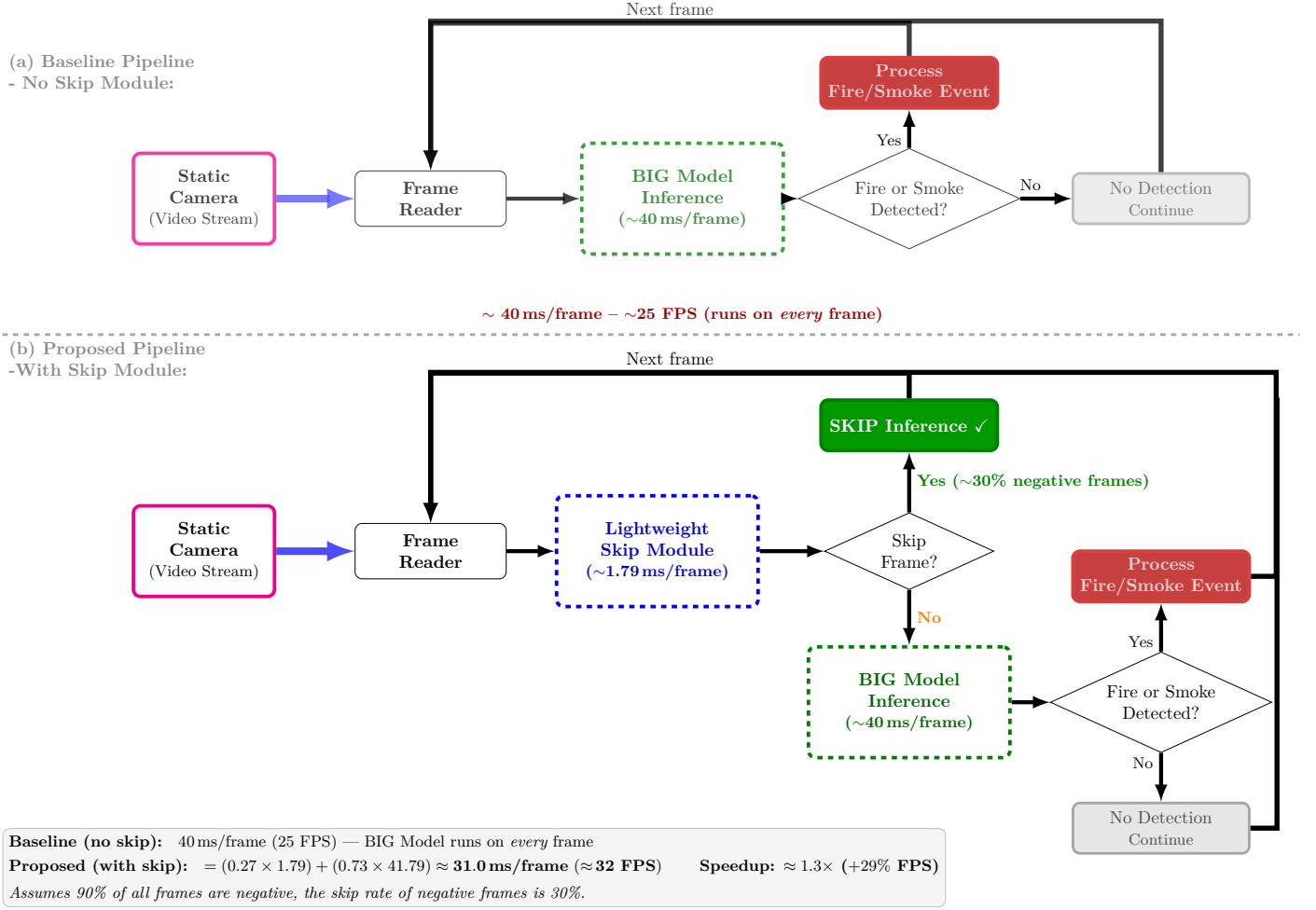


Figure 1: Conventional frame-wise inference pipeline (top) versus the proposed skip-module-enabled pipeline (bottom) for indoor fire and smoke detection.

The proposed system is a fire and smoke detection pipeline that processes a continuous video stream and classifies each frame as either containing fire/smoke or not. The proposed framework augments the conventional frame-wise inference pipeline with a lightweight skip module $\mathcal{S}$, which serves as a gating mechanism that selectively suppresses unnecessary DL inference on frames with low likelihood of containing fire or smoke — most commonly, static background frames — thereby reducing redundant computation and improving overall throughput. Formally, let f_t denote the video frame at time step t , $\mathcal{M}$ the DL classifier, and $\hat{y}_t \in \{0, 1\}$ the predicted label for f_t .

Baseline system. In this baseline, every frame f_t is unconditionally forwarded to $\mathcal{M}$, which outputs a binary prediction $\hat{y}_t \in \{0, 1\}$ indicating the presence or absence of fire or smoke, as follows:

$$\hat{y}_t = \mathcal{M}(f_t)$$

The classifier operates at the full frame rate without any pre-filtering, processing each frame independently of scene content. This per-frame inference pipeline serves as the performance reference when evaluating the proposed skip module.

Proposed system. The skip module $\mathcal{S}$ first evaluates f_t based on inter-frame motion cues and outputs a binary gate decision s_t . While this work instantiates $\mathcal{S}$ solely motion-based cues, the formulation is general and can incorporate other types of cues such as color, texture, or learned features. The resulting system output is given by:

$$\hat{y}_t = \begin{cases} \mathcal{M}(f_t) & \text{if } s_t = 1 \quad (\text{run inference}) \\ 0 & \text{if } s_t = 0 \quad (\text{skip, label as negative}) \end{cases}$$

In this work, skip module $\mathcal{S}$ derives s_t from the motion estimated between f_t and the previous frame f_{t-1} . Frames with little or no motion are unlikely to contain fire or smoke, and are therefore skipped without invoking the classifier $\mathcal{M}$. The classifier $\mathcal{M}$ is typically a high-capacity and, accurate classifier but it is computationally expensive. In contrast, $\mathcal{S}$ is deliberately designed to be computationally lightweight, introducing negligible overhead. The objective of $\mathcal{S}$ is to skip as many non-event frames as possible while minimizing missed fire and smoke detections, thereby improving overall system throughput (FPS)

while maintaining high detection accuracy. The architecture and operation of skip module $\mathcal{S}$ are described in the following subsection.

3.2 Skip Module Design

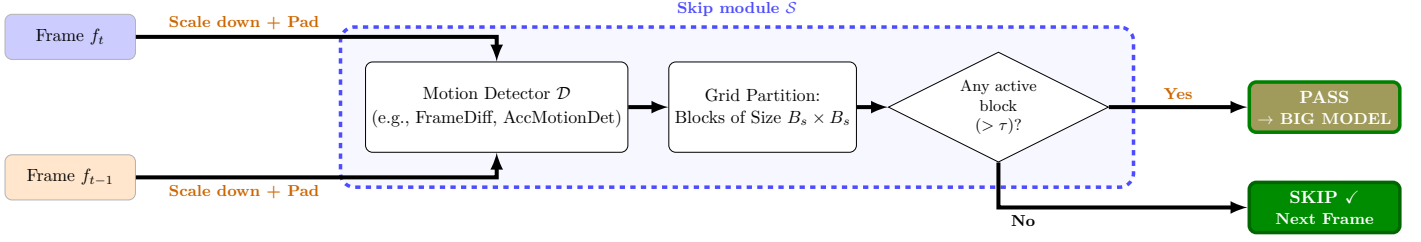


Figure 2: Skip Module for Fire and Smoke Detection

Algorithm 1 Skip Module $\mathcal{S}$: Motion-Based Frame Gating

Input: Video stream $\{f_1, f_2, \dots\}$, motion detector $\mathcal{D}$,

Parameters: scale factor α , block size B , block active threshold τ

Output: Skip decisions $\{s_1, s_2, \dots\}$, where $s_t \in \{0, 1\}$

- 1: $\tilde{f}_{\text{prev}} \leftarrow \text{null}$ $\triangleright$ Initialize previous downsampled frame
 - 2: **for** each incoming frame f_t **do**
 - Step 1: Pre-process**
 - 3: Downsample f_t by α to obtain $\tilde{f}_t$
 - 4: Let $B_s \leftarrow B \cdot \alpha$ $\triangleright$ Effective block size on downsampled frame
 - 5: Pad $\tilde{f}_t$ to be divisible by B_s
 - Step 2: Compute Foreground Mask**
 - 6: **if** $\tilde{f}_{\text{prev}} = \text{null}$ **then** $\triangleright$ No previous frame on first iteration
 - 7: $s_t \leftarrow 1$ $\triangleright$ Run inference on first frame
 - 8: $\tilde{f}_{\text{prev}} \leftarrow \tilde{f}_t$
 - 9: **continue**
 - 10: **end if**
 - 11: $\mathbf{M}_t \leftarrow \mathcal{D}(\tilde{f}_t, \tilde{f}_{\text{prev}})$ $\triangleright \mathcal{D}$: e.g., FrameDiffDet or AccMotionDet
 - 12: Partition $\mathbf{M}_t$ into a grid of non-overlapping blocks of size B_s
 - 13: Mark a block as *active* if its foreground pixel ratio $> \tau$
 - Step 3: Gating Decision**
 - 14: **if** any active block exists **then**
 - 15: $s_t \leftarrow 1$ $\triangleright$ Motion detected, run DL inference
 - 16: **else**
 - 17: $s_t \leftarrow 0$ $\triangleright$ No motion, skip inference
 - 18: **end if**
 - 19: $\tilde{f}_{\text{prev}} \leftarrow \tilde{f}_t$ $\triangleright$ Update previous frame for next iteration
 - 20: **yield** s_t
 - 21: **end for**
-

The proposed skip module $\mathcal{S}$ is designed to suppress redundant DL inference by selectively identifying informative frames in the continuous video stream. As illustrated in Figure 2, each input frame f_t is first downsampled by a factor of α and padded to ensure its dimensions are divisible by the effective block size $B_s = B \cdot \alpha$, where B is the block size defined on the original frame. The motion detector $\mathcal{D}$ then computes a binary foreground mask from consecutive frames, which is partitioned into non-overlapping blocks of size $B_s \times B_s$. If the fraction of active pixels in any block exceeds the block active threshold τ , the skip module sets $s_t = 1$ and forwards f_t to the classifier $\mathcal{M}$; otherwise, the frame is skipped ($s_t = 0$). The full procedure is formalized in Algorithm 1.

As illustrated in Figure 2 and formalized in Algorithm 1, the skip module $\mathcal{S}$ can leverage any available information — such as motion, color, or texture — to compute the skip decision s_t . In this work, we focus on motion information derived from consecutive frames as the primary cue. Specifically, we design and evaluate two motion-based instantiations of $\mathcal{D}$: (1) *FrameDiffDet* — a lightweight method that estimates motion via direct inter-frame differencing, and (2) *AccMotionDet* — an extension that accumulates motion evidence over consecutive frames for a more robust skip decision. Both approaches are selected for their simplicity and low computational cost, making them well-suited for real-time deployment. The following subsections describe each method in detail, beginning with the simpler FRAMEDIFFDET before introducing the accumulation-based ACCMOTIONDET.

3.2.1 FrameDiffDet — Naive Motion Detection

Let F_t and F_{prev} denote the grayscale frames converted from the downsampled inputs $\tilde{f}_t$ and $\tilde{f}_{\text{prev}}$, respectively. The detector takes these two consecutive downsampled frames as input and produces a binary foreground mask $\mathbf{M}_t \in \{0, 255\}$ as output, where each pixel is marked active (255) if the absolute inter-frame intensity difference exceeds a sensitivity threshold τ_d , and inactive (0) otherwise. The procedure is formalized in Algorithm 2.

Algorithm 2 Frame Difference Detector (FrameDiffDet)

Input: Downsampled frames $\tilde{f}_t, \tilde{f}_{\text{prev}}$

Parameters: τ_d : pixel difference sensitivity threshold

Output: $\mathbf{M}_t \in \{0, 255\}$: binary foreground mask

- 1: Convert $\tilde{f}_t$ and $\tilde{f}_{\text{prev}}$ to grayscale frames F_t and F_{prev}
 - 2: $\mathbf{M}_t(i, j) \leftarrow \begin{cases} 255 & |F_t(i, j) - F_{\text{prev}}(i, j)| > \tau_d \\ 0 & \text{otherwise} \end{cases}$
 - 3: **return** $\mathbf{M}_t$
-

3.2.2 AccMotionDet — Motion Detection with Accumulation

Inspired by [16], we design ACCMOTIONDET to address the key limitation of FRAMEDIFFDET: its reliance on a single inter-frame difference makes it sensitive to noise and single-frame flicker, which may trigger unnecessary inference calls. Instead, ACCMOTIONDET accumulates motion evidence over multiple consecutive frames, providing a more robust signal for motion estimation and enabling more accurate skip decisions.

Let F_t and F_{prev} denote the grayscale frames converted from the downsampled frames $\tilde{f}_t$ and $\tilde{f}_{\text{prev}}$, respectively. Let $\mathbf{K}_{t-1} \in [0, K_{\text{max}}]$ denote the per-pixel *accumulated motion mask* from the previous frame. The detector takes two consecutive downsampled frames and $\mathbf{K}_{t-1}$ as input, and produces a binary foreground mask $\mathbf{M}_t \in \{0, 255\}$ and the updated mask $\mathbf{K}_t$ as output. $\mathbf{K}_t$ is incremented by ω when the inter-frame pixel difference exceeds τ_d , and decays by δ each frame, bounded at K_{max} to prevent unbounded growth under persistent motion. A pixel is marked active (255) only when $\mathbf{K}_t \geq \tau_m$, suppressing transient noise and single-frame flicker, and inactive (0) otherwise. The procedure is formalized in Algorithm 3.

Algorithm 3 Accumulation Motion Detector (AccMotionDet)

Input: Downsampled frames $\tilde{f}_t, \tilde{f}_{\text{prev}}$; accumulated mask $\mathbf{K}_{t-1}$ (initialized to $\mathbf{0}$)

Parameters: τ_d : pixel difference sensitivity, ω : motion increment, τ_m : activation threshold, K_{max} : accumulation cap, δ : decay rate

Output: $\mathbf{M}_t \in \{0, 255\}$: binary foreground mask; $\mathbf{K}_t$: updated accumulated mask

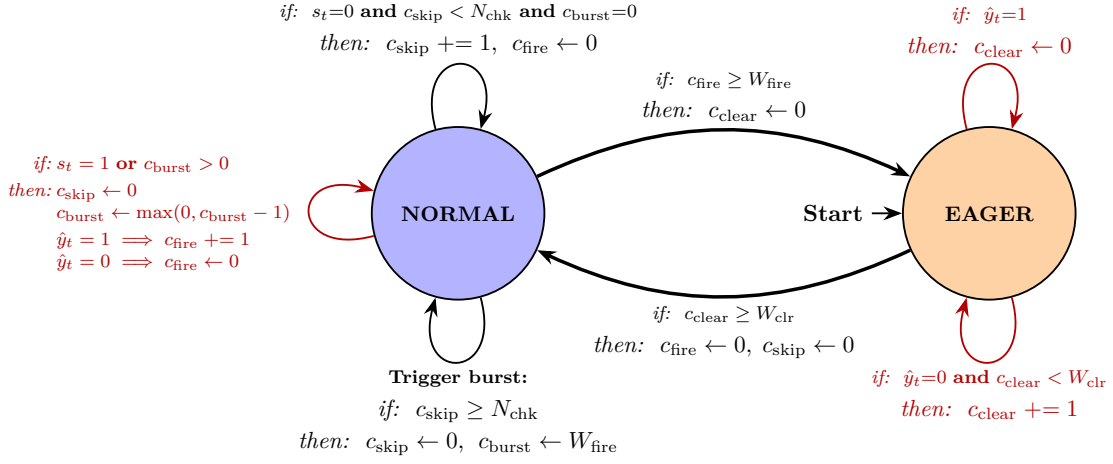
- 1: Convert $\tilde{f}_t$ and $\tilde{f}_{\text{prev}}$ to grayscale frames F_t and F_{prev}
 - 2: $\mathbf{D}_t \leftarrow |F_t - F_{\text{prev}}|$ ▷ Absolute inter-frame difference
 - 3: $\Delta_t(i, j) \leftarrow \begin{cases} \omega & \mathbf{D}_t(i, j) > \tau_d \\ 0 & \text{otherwise} \end{cases}$ ▷ Instantaneous increment map
 - 4: $\mathbf{K}_t \leftarrow \min(\mathbf{K}_{t-1} + \Delta_t, K_{\text{max}})$ ▷ Accumulate and cap
 - 5: $\mathbf{K}_t \leftarrow \max(\mathbf{K}_t - \delta, 0)$ ▷ Temporal decay
 - 6: $\mathbf{M}_t(i, j) \leftarrow \begin{cases} 255 & \mathbf{K}_t(i, j) \geq \tau_m \\ 0 & \text{otherwise} \end{cases}$ ▷ Threshold to binary mask
 - 7: **return** $\mathbf{M}_t, \mathbf{K}_t$
-

3.3 Eager Mode Design

A preliminary analysis of the self-constructed indoor surveillance video dataset revealed that slow-developing or nearly stationary smoke events constitute a key challenge for the skip module. In such cases, the subtle inter-frame motion associated with these events may fall below the activation threshold of the motion detector, causing the skip module to incorrectly bypass frames containing actual fire or smoke activity. To mitigate this limitation, the proposed framework augments the skip module with an EAGER mode that periodically forces inference updates.

When the classifier $\mathcal{M}$ produces positive fire/ smoke predictions on W_{fire} consecutive non-skipped frames, the system transition from Normal mode to EAGER mode and suppresses all skip decisions by setting $s_t = 1$, forcing inference on every frame. The system remains in EAGER mode until the scene is declared safe again, which occurs when the classifier $\mathcal{M}$ returns negative predictions for W_{clr} consecutive frames. At that point, the system returns to NORMAL mode with the skip module re-enabled.

Additionally, while operating in Normal mode, the system periodically forces inference on W_{fire} consecutive frames after every N_{chk} skipped frames. This mechanism ensures the classifier retains opportunities to detect emerging fire or smoke activity



Operational Modes	
■ NORMAL	Skip Module enabled. Model $\mathcal{M}$ is invoked only if motion is detected ($s_t = 1$) or a forced burst is active.
■ EAGER	Continuous inference. Skip module $\mathcal{S}$ is bypassed; $\mathcal{M}$ evaluates every frame. (Default starting state).
Transition Types	
→	Inference executed: $\mathcal{M}(f_t)$ is invoked to evaluate the current frame.
$\rightarrow$	Zero-inference step: Frame is skipped, or state variables are updated seamlessly.
Variables & Parameters	
$s_t \in \{0, 1\}$	Skip module ($\mathcal{S}$) prediction (1: motion, 0: static).
$\hat{y}_t \in \{0, 1\}$	Classification model ($\mathcal{M}$) prediction (1: fire/smoke, 0: safe).
c_{fire}	Counter for consecutive positive detections ($\hat{y}_t = 1$).
c_{clear}	Counter for consecutive safe frames ($\hat{y}_t = 0$) during EAGER mode.
c_{skip}	Counter for consecutively skipped frames during NORMAL mode.
c_{burst}	Countdown variable for remaining frames in a forced inference burst.
W_{fire}	Threshold of consecutive positive detections required to trigger EAGER mode.
W_{clr}	Threshold of consecutive negative frames required to return to NORMAL mode.
N_{chk}	Maximum allowed skipped frames before triggering a forced burst.

Figure 3: State machine of the EAGER mode extension to the ACCMOTIONDET-based skip module. Transitions are annotated with *if* conditions and *then* actions specifying variable updates. In NORMAL mode, $\mathcal{S}$ operates as designed, with periodic forced checks every N_{chk} frames. Upon W_{fire} consecutive positive predictions, the system enters EAGER mode, bypassing $\mathcal{S}$ (i.e., $s_t = 1$) to forward every frame to $\mathcal{M}$. The system returns to NORMAL mode after W_{clr} consecutive negative predictions.

and transition to EAGER mode when necessary. Consequently, the proposed design improves recall for slow-developing or nearly stationary smoke events while preserving the throughput gains provided by the skip module during NORMAL mode operation.

Figure 3 illustrates the state machine of EAGER mode with the transition conditions and corresponding actions for each state and how states transition during system operation.

3.4 Skip Module Hyperparameter Optimization

The skip module $\mathcal{S}$, as described in Algorithm 1, is governed by several hyperparameters — including scale factor α , block active threshold τ , and pixel difference sensitivity threshold τ_d — that can significantly affect overall system performance. To identify the optimal configuration, we employ a constrained grid search over a validation set D_{val} . Let R_{base} denote the baseline recall without skipping. For each candidate $\theta \in \Theta$, we evaluate the full pipeline (comprising the skip module $\mathcal{S}(\theta)$ followed by the classifier $\mathcal{M}$) on D_{val} and compute recall $R(\theta)$ and skip rate $S_r(\theta)$ (see Section 4.2 for formal definitions).

Feasibility constraint. A candidate θ is feasible only if:

$$R(\theta) \geq R_{\text{base}} - \delta_R, \quad (1)$$

where $\delta_R > 0$ is the maximum allowable recall drop (e.g., $\delta_R = 0.01$ permits at most a 1% reduction relative to the baseline).

Scoring and selection. For each feasible candidate, we compute the recall retention score

$$\rho(\theta) = 1 - \frac{R_{\text{base}} - R(\theta)}{\delta_R}, \quad (2)$$

which maps the recall drop linearly onto $[0, 1]$ within the feasible region. Both $\rho(\theta)$ and $S_r(\theta)$ are range-normalized over Θ_{feasible} (Pass 2 of Algorithm 4), yielding $\tilde{\rho}(\theta)$ and $\tilde{S}_r(\theta) \in [0, 1]$, so that the weights w_S and w_R directly reflect their intended

Algorithm 4 Skip Module Hyperparameter Selection

Input: Parameter space Θ , validation set D_{val} , DL model $\mathcal{M}$ **Parameters:** recall tolerance δ_R , weights w_S, w_R **Output:** Optimal parameters θ^*

```
1: baseline  $\leftarrow$  EVALUATEPIPELINE( $D_{\text{val}}$ , skip=None,  $\mathcal{M}$ )
2: Extract  $R_{\text{base}}$  from baseline
3:  $\Theta_{\text{feasible}} \leftarrow \emptyset$ 
```

▷ Pass 1: Collect feasible candidates

```
4: for each  $\theta \in \Theta$  do
5:   results  $\leftarrow$  EVALUATEPIPELINE( $D_{\text{val}}$ ,  $\mathcal{S}(\theta)$ ,  $\mathcal{M}$ )
6:   Extract  $R(\theta)$ ,  $S_r(\theta)$  from results
7:   if  $R(\theta) \geq R_{\text{base}} - \delta_R$  then
8:      $\rho(\theta) \leftarrow 1 - \frac{R_{\text{base}} - R(\theta)}{\delta_R}$ 
9:      $\Theta_{\text{feasible}} \leftarrow \Theta_{\text{feasible}} \cup \{(\theta, \rho(\theta), S_r(\theta))\}$ 
10:  end if
11: end for
```

▷ Pass 2: Range-normalize and select

```
12:  $\tilde{\rho}(\theta) \leftarrow \frac{\rho(\theta) - \min_{\Theta_{\text{feasible}}} \rho}{\max_{\Theta_{\text{feasible}}} \rho - \min_{\Theta_{\text{feasible}}} \rho}$  for each  $\theta \in \Theta_{\text{feasible}}$ 
13:  $\tilde{S}_r(\theta) \leftarrow \frac{S_r(\theta) - \min_{\Theta_{\text{feasible}}} S_r}{\max_{\Theta_{\text{feasible}}} S_r - \min_{\Theta_{\text{feasible}}} S_r}$  for each  $\theta \in \Theta_{\text{feasible}}$ 
14:  $\theta^* \leftarrow \arg \max_{\theta \in \Theta_{\text{feasible}}} [w_R \tilde{\rho}(\theta) + w_S \tilde{S}_r(\theta)]$ 
15: return  $\theta^*$ 
```

relative importance. The optimal configuration is then:

$$\theta^* = \arg \max_{\theta \in \Theta_{\text{feasible}}} [w_R \tilde{\rho}(\theta) + w_S \tilde{S}_r(\theta)], \quad (3)$$

where $\Theta_{\text{feasible}} = \{\theta \in \Theta : R(\theta) \geq R_{\text{base}} - \delta_R\}$ and $w_S + w_R = 1$.

FAR guarantee and exclusion from scoring. Let $FP(\theta)$ and FP_{base} denote the number of false positives produced by the skip-enabled and baseline systems, respectively, and let $N_{\text{skip_neg}}(\theta)$ denote the number of ground-truth negative frames skipped by the skip module $\mathcal{S}$ under configuration θ (see Section 4.2 for definitions of FAR, S_r , and N_{neg}). Since every skipped frame is labeled negative, false positives arise only from frames passed to classifier $\mathcal{M}$, so $FP(\theta) = FP_{\text{base}} - a$ for some $a \geq 0$. Because a skipped false positive must simultaneously be a baseline false positive and a skipped negative frame, $a \leq \min(FP_{\text{base}}, N_{\text{skip_neg}})$ without any assumption on the relationship between the skip module $\mathcal{S}$ and the classifier $\mathcal{M}$. Dividing by N_{neg} and substituting $\text{FAR}_{\text{base}} = FP_{\text{base}}/N_{\text{neg}}$ and $S_r(\theta) = N_{\text{skip_neg}}/N_{\text{neg}}$ yields the assumption-free interval:

$$\text{FAR}(\theta) \in [\max(0, \text{FAR}_{\text{base}} - S_r(\theta)), \text{FAR}_{\text{base}}]. \quad (4)$$

The upper bound confirms that the skip module cannot increase the false alarm rate by design; the primary safety risk is therefore recall degradation caused by incorrectly skipped fire/smoke frames, which motivates enforcing the recall constraint as a hard feasibility gate. Explicit FAR optimization is nonetheless unnecessary: since $\mathcal{S}$ operates solely on inter-frame motion without access to $\mathcal{M}$'s outputs, its hyperparameters do not directly control which baseline false positives are removed, and including FAR in the objective could favor configurations that overfit to validation-specific error patterns. We accordingly report $\text{FAR}(\theta)$ only as an observed consequence metric in the results tables.

4 Experiments and Results

4.1 Fire and Smoke Indoor Video Dataset

Several existing benchmarks address this detection task, including FireNet [17], Firesense [18], FiSmo [19], and FURG [20]. However, these were recorded with moving cameras and are therefore unsuitable for static surveillance scenarios. Static-camera datasets such as DFire [21], VisiFire Bilkent [22], KMU Fire and Smoke [23], Mivia Fire and Mivia Smoke datasets [24], and USTC Smoke [25] do exist; however, they collectively suffer from several limitations: a predominance of outdoor scenes, a small number of video samples, low spatial resolution, and insufficient diversity in fire/smoke appearances and environmental conditions.

To address these deficiencies, we constructed a dedicated static indoor video dataset by aggregating clips from multiple heterogeneous sources. Fire and smoke samples were sourced from the Korea AI Fire Dataset [26], the USTC Smoke Dataset [25], and the VDS3 Dataset [27], supplemented by videos collected from open online platforms (Pexels, Pixabay, and YouTube). Non-fire/smoke (negative) samples were compiled from self-recorded footage captured by real CCTV cameras in various indoor environments (e.g., parking areas), along with videos from the Safe & Unsafe Behavior in Workplaces

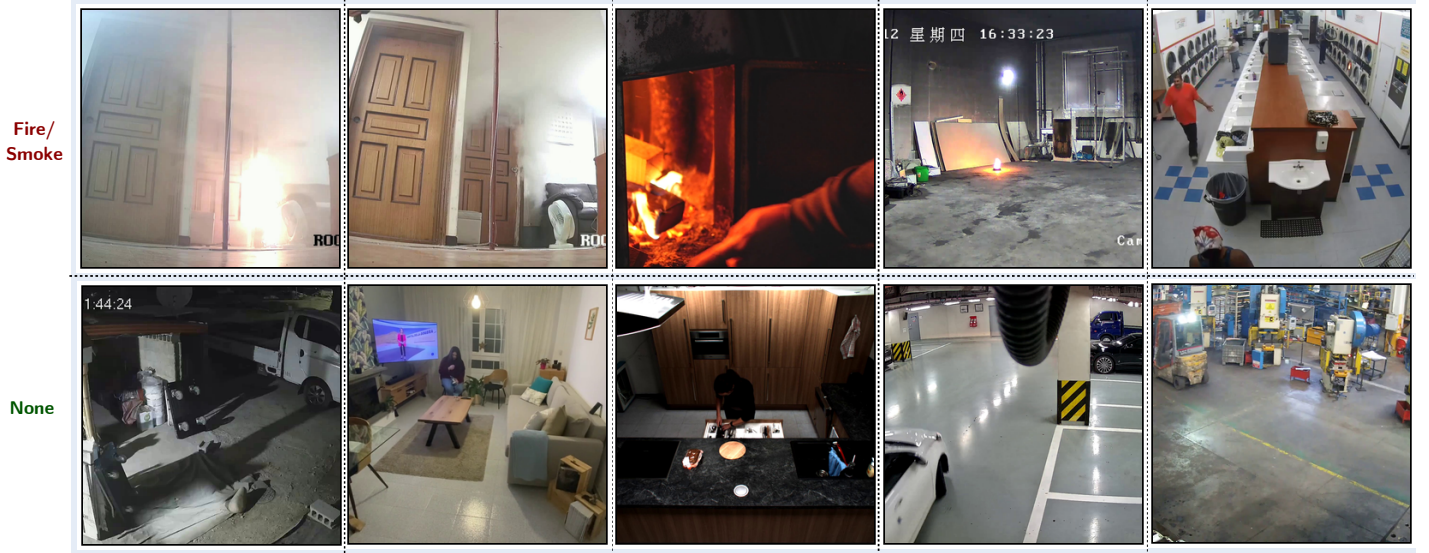


Figure 4: Representative frames extracted from the indoor fire and smoke video dataset used in this study.

Table 1: Comparison of existing fire and smoke video datasets with the proposed dataset (150 videos; static CCTV cameras; diverse indoor environments). Unlike most existing datasets, 143 out of 150 videos are recorded at HD resolution (1280×720) or above, matching commercial surveillance camera standards.

Model	Category	Videos	Resolution Min	Resolution Max	FireSmoke	Normal	Indoor	Outdoor
FireSense (2010) [18]	Moving camera	49	300×240	1600×1200	24	25	10	39
KMU (2011) [23]	Static camera	38	320×240	320×240	28	10	10	28
VisiFireBilkent (2015) [22]	Static camera	39	320×240	720×576	38	1	6	32
Mivia FIRE (2015) [24]	Static camera	31	320×240	800×600	28	3	4	27
Mivia SMOKE (2015) [24]	Static camera	149	292×240	292×240	76	73	0	149
FURG (2015) [20]	Moving camera	22	240×344	1920×1080	17	5	3	19
USTC Smoke (2017) [25]	Static camera	30	1920×1080	1920×1080	22	8	30	0
FireNet (2019) [17]	Moving camera	62	352×222	1920×1080	46	16	30	32
FiSmo (2020) [19]	Moving camera	88	320×240	1920×1080	80	8	0	88
D-Fire (2021) [21]	Static camera	100	1280×720	1280×720	50	50	0	100
VDS3 (2022) [27]	Mix camera	20	352×288	2048×1536	8	12	6	14
Ours (2026) - This work	Static camera	150	814×720	3840×2160	75	75	150	0

dataset [28], the Indoor Action dataset [29], the MPII Cooking 2 Dataset [30], and the WiseNet dataset [31]. Table 1 summarizes the properties of the self-collected video dataset alongside a comparison with existing datasets, and Figure 4 presents representative sample frames from our dataset.

For the hyperparameter optimization described in Section 3.4, the video dataset was further partitioned into a validation set, D_{val} (46 videos), used to select the optimal skip-module hyperparameters, and a test set, D_{test} (104 videos), used for the final evaluation of the proposed system against the baseline and existing methods. The partition adhered to a 30:70 ratio (validation:test), following the protocol of [10], and was performed using stratified sampling to preserve balanced class distributions in both subsets.

4.2 Evaluation Metrics

Performance is evaluated under **frame-level** evaluation, following the standard protocol for fire and smoke detection in videos [32]. Frame-level evaluation measures detection performance for each individual frame, providing a strict assessment of classification accuracy. The following label definitions apply to this evaluation protocol:

- True Positive (TP): Correct detection of fire or smoke.
- True Negative (TN): Correct identification of the absence of fire or smoke.
- False Positive (FP): Incorrect detection of fire or smoke when none is present.
- False Negative (FN): Failure to detect fire or smoke when present.

Quantitative results are reported using standard classification metrics — accuracy, recall, false alarm rate (FAR), precision, F1-score, and frames per second (FPS) — as defined below.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad (5)$$

$$\text{Recall (R)} = \frac{TP}{TP + FN}, \quad (6)$$

$$\text{False Alarm Rate (FAR)} = \frac{FP}{FP + TN}, \quad (7)$$

$$\text{Precision (P)} = \frac{TP}{TP + FP}, \quad (8)$$

$$\text{F1-score} = 2 \times \frac{P \times R}{P + R}, \quad (9)$$

$$\text{FPS} = \frac{\text{Total Frames}}{\text{Total Processing Time (seconds)}}, \quad (10)$$

$$(11)$$

where TP , TN , FP , and FN represent True Positive, True Negative, False Positive, and False Negative, respectively.

In addition to standard metrics, we report the skip rate S_r , a system-level efficiency criterion specific to the proposed skip-based framework. Skip rate measures the proportion of ground-truth negative frames correctly bypassed by the skip module $\mathcal{S}$, defined as

$$S_r = \frac{N_{\text{skip_neg}}}{N_{\text{neg}}} \quad (12)$$

where $N_{\text{skip_neg}}$ is the number of ground-truth negative frames assigned $s_t = 0$ by the skip module $\mathcal{S}$ (i.e., background frames bypassed without invoking big DL model $\mathcal{M}$), and N_{neg} is the total number of ground-truth negative frames in the evaluated set. A higher S_r indicates greater computational savings, while a drop in recall may signal unsafe skipping of fire/smoke frames.

4.3 Models and Implementation Details

Baseline Models: To demonstrate both the superiority of the high-capacity classifier over lightweight alternatives and the effectiveness of the proposed skip module in accelerating its inference, we evaluate the following baseline models alongside our BIG Model:

- **MobileNet** [33]: A modified MobileNet architecture fine-tuned for fire and smoke detection.
- **FireNet** [17]: A lightweight CNN-based image classifier comprising 14 layers, with a model size of 7.5 MB and approximately 650k trainable parameters.
- **YOLOv5s** [10]: A lightweight object detector based on YOLOv5 [34], trained with hyperparameters selected via grid search on the D-Fire dataset [21].
- **YOLOv5l** [10]: A heavier variant of the above, based on the larger YOLOv5l architecture, offering higher capacity at increased computational cost.

Proposed Classifier (BIG Model $\mathcal{M}$ — HGNetV2): The BIG Model $\mathcal{M}$ is obtained by fine-tuning a pretrained High-Performance GPU Network v2 (HGNetV2), specifically the `hgnetv2_b5.ssl_d_stage2_ft_in1k` checkpoint [35] from the Timm library [36], which was originally trained using SSLD knowledge distillation [37]. HGNetV2 [38] is a high-capacity CNN architecture designed to achieve substantially higher accuracy than models of comparable inference speed on NVIDIA GPUs, making it well-suited for real-life deployment. For fine-tuning, a dataset of 253,325 fire/smoke/normal images was compiled from internet sources and partitioned into training (90%) and validation (10%) subsets. The model was trained for 100 epochs with a batch size of 64 using the Adam optimizer [39], configured with a learning rate of 0.003, weight decay of 0.05, and a warmup over the first 5 epochs. The training was conducted on a single NVIDIA RTX 5090 GPU (32 GB VRAM) paired with an Intel Core Ultra 9 285K processor and 128 GB of system RAM, running PyTorch 2.7.1 with CUDA Toolkit 12.8. All input images were preprocessed by resizing to $3 \times 360 \times 640$ ($C \times H \times W$). The fine-tuned model achieved a classification accuracy of 98.34% and a high recall of 98.12% on the validation set.

Implementation Details: Unless stated otherwise, all experiments were conducted on a workstation equipped with an Intel Core i9-10900K CPU, 64 GB DDR4 system memory, and an NVIDIA GeForce RTX 3090 GPU (24 GB VRAM), running Windows 10 Pro (22H2, build 19045). Deep learning inference was performed using PyTorch 2.7.1 under CUDA 12.9, and video processing and motion estimation were carried out using OpenCV 4.11 (CPU-only).

4.4 Results

4.4.1 Hyperparameter Optimization Results

In this work, the recall tolerance δ_R is set to $\delta_R = 0.015$, permitting a maximum absolute recall drop of 1.5% relative to the baseline. At the baseline recall of approximately 95%, this ensures the worst-case deployed system retains a recall of

at least 93.5%, while affording the hyperparameter search sufficient flexibility to identify configurations with meaningful efficiency gains. The optimization weights are set to $w_S = 0.7$ and $w_R = 0.3$, prioritizing skip rate (the primary determinant of throughput gain) while using recall retention as a secondary criterion to favor configurations closer to baseline performance among feasible candidates.

4.4.1.1 Hyperparameter Optimization Results for the FrameDiffDet-based Skip Module To identify the optimal configuration for the FrameDiffDet-based skip module, we performed a systematic grid search over four hyperparameters: scale factor α , block size B , block active threshold τ , and difference sensitivity threshold τ_d , yielding a total of $2 \times 2 \times 3 \times 4 = 48$ candidate configurations. The search space is as follows:

- **Scale factor** $\alpha \in \{0.5, 1.0\}$: controls the downsampling ratio applied to input frames prior to motion computation. A smaller α reduces computational cost but may lose subtle motion detail. For simplicity, we evaluate full resolution ($\alpha = 1.0$) and half resolution ($\alpha = 0.5$).
- **Block size** $B \in \{16, 32\}$: determines the spatial granularity of motion detection, expressed in pixels of the original frame (the effective block size on the downsampled frame is $B_s = B \cdot \alpha$). Smaller blocks ($B = 16$) capture localized motion from small fire or smoke regions, while larger blocks ($B = 32$) aggregate motion over a broader spatial context, providing greater robustness against isolated pixel-level noise.
- **Block active threshold** $\tau \in \{0.05, 0.10, 0.15\}$: defines the minimum foreground pixel ratio within a block for it to be declared active; inference is triggered if at least one active block is detected. A low value ($\tau = 0.05$) is sensitive to sparse motion within a block, minimizing missed detections, while a higher value ($\tau = 0.15$) demands denser local motion, suppressing responses to minor pixel-level disturbances. The three values provide a systematic sweep from fine to coarse sensitivity.
- **Pixel difference sensitivity threshold** $\tau_d \in \{3, 5, 7, 10\}$: defines the minimum absolute inter-frame intensity change required for a pixel to be declared active. This range is chosen to cover diverse sensitivity levels, from near-noise-level detection ($\tau_d = 3$, responding to subtle illumination changes) to robust large-motion detection ($\tau_d = 10$, requiring strong, unambiguous motion).

Table 2 presents representative configurations illustrating the recall–efficiency trade-off; the full search space of 48 configurations is omitted for brevity. The results reveal a fundamental limitation of FRAMEDIFFDET: it fails to deliver meaningful throughput improvement under the safety constraint $\delta_R = 0.015$.

Among all 48 evaluated configurations, only three satisfy the recall constraint, all requiring the lowest sensitivity setting $\tau_d = 3$. The best-ranked feasible configuration ($\alpha=0.5$, $B=16$, $\tau=0.05$, $\tau_d=3$) achieves a skip rate of merely 0.94%, bypassing fewer than 1 in 100 background frames. Consequently, the FPS of all feasible configurations (22.02–23.32) falls below the no-skip baseline (24.66), indicating that the skip module overhead outweighs the benefit of skipped inference calls. Conversely, configurations that do achieve substantial skip rates (reaching 47–60% at higher τ_d values of 7 and 10) suffer severe recall degradation of 5.5–10%, far exceeding the safety tolerance.

These results demonstrate that FRAMEDIFFDET, without noise suppression or robust sustained-motion confirmation, is unsuitable as a skip gate: it either skips too few frames to yield any efficiency gain, or skips too aggressively and suffer unacceptable recall loss. This motivates the development of ACCMOTIONDET, which accumulates motion evidence over multiple frames to provide a more reliable signal for skip decisions, as described in the following section.

4.4.1.2 Hyperparameter Optimization Results for the AccMotionDet-based Skip Module The hyperparameter search for ACCMOTIONDET is designed to mirror the FrameDiffDet study while accounting for the temporal accumulation of motion scores over consecutive frames. Details of the search space (of 96 configurations) are as follows:

- **Scale factor** ($\alpha \in \{0.5, 1.0\}$) and **Block size** ($B \in \{16, 32\}$): Retained from FrameDiffDet to maintain consistency in the spatial resolution and grid density trade-offs.
- **Block active threshold** ($\tau \in \{0.05, 0.10\}$): Restricted to a narrower range than in FrameDiffDet, as temporal accumulation naturally suppresses false activations, rendering more aggressive thresholds unnecessary.
- **Pixel difference sensitivity threshold** ($\tau_d \in \{3, 5\}$): Limited to the lower end of the FrameDiffDet search space, as higher values were found to cause excessive frame skipping and unacceptable recall degradation, as described in Section 4.4.1.1.
- **Accumulation step** ($\omega = 5$) and **Decay rate** ($\delta = 1$): We fix these parameters to streamline the hyperparameter search, allowing us to focus on the primary sensitivity controls: the activation threshold (τ_m) and accumulation cap ($K_{\max}$). Since the optimal tuning of τ_m and $K_{\max}$ depends directly on ω , keeping ω constant provides a stable baseline for optimization. Furthermore, setting $\delta = 1$ ensures consistent, linear decay behavior while reducing the dimensionality of the search space.
- **Activation threshold** ($\tau_m \in \{5, 10\}$): This activation threshold labels pixels in the motion mask as active only when their accumulated motion score reaches or exceeds τ_m . Given the fixed accumulation parameters ($\omega = 5, \delta = 1$), $\tau_m = 5$ and $\tau_m = 10$ are approximately equivalent to requiring motion signals over 2 and 3 consecutive frames, respectively.
- **Accumulation cap** ($K_{\max} \in \{15, 25, 35\}$): This parameter limits the maximum accumulated motion score to prevent runaway accumulation during prolonged activity. Without this cap, prolonged or intense motion could drive the

Table 2: Representative FRAMEDIFFDET hyperparameter configurations evaluated on the validation set, illustrating the recall–efficiency trade-off across the full search space of 48 configurations (subset shown for brevity). Configurations with a ranking score of “-” violate the recall constraint ($\delta_R = 0.015$) and are included solely to illustrate the trade-off at higher skip rates. Recall (%) reports the retained recall with the absolute drop relative to the baseline in parentheses, i.e., $R(-\Delta R)$. The best and second-best values per metric are highlighted in **bold** and *italic underline*, respectively.

Exp.	Parameters				Metrics				Ranking Score
	Scale factor α	Block size B	Block active threshold τ	Diff threshold τ_d	Recall (%) $\uparrow$	Skip Rate (%) $\uparrow$	FAR (%) $\downarrow$	FPS $\uparrow$	
Baseline (No Skip module)	-	-	-	-	94.347 (-0.000)	0.0	0.74	24.66	-
FrameDiff	0.5	16	0.05	3	93.971 (-0.377)	0.94	0.737	23.32	0.8046
FrameDiff	1.0	16	0.1	3	93.539 (-0.809)	0.97	0.737	22.26	<u>0.7000</u>
FrameDiff	1.0	16	0.05	3	<u>94.176</u> (-0.171)	0.72	0.737	22.02	0.3000
FrameDiff	0.5	16	0.1	3	92.833 (-1.514)	1.35	0.737	23.60	-
FrameDiff	1.0	16	0.15	3	92.052 (-2.295)	1.53	0.737	22.53	-
FrameDiff	1.0	32	0.05	3	91.627 (-2.720)	2.43	0.737	23.16	-
FrameDiff	1.0	16	0.05	5	91.463 (-2.885)	1.94	0.737	22.60	-
FrameDiff	0.5	16	0.15	3	91.449 (-2.898)	3.92	0.737	24.27	-
FrameDiff	0.5	32	0.05	3	90.983 (-3.364)	7.11	0.737	25.32	-
FrameDiff	0.5	16	0.05	5	90.552 (-3.796)	6.23	0.737	24.68	-
FrameDiff	0.5	16	0.1	7	88.873 (-5.474)	47.22	<u>0.733</u>	37.34	-
FrameDiff	0.5	16	0.05	10	88.188 (-6.160)	<u>49.0</u>	<u>0.733</u>	<u>38.31</u>	-
FrameDiff	0.5	32	0.05	10	84.556 (-9.791)	59.53	0.73	45.54	-

Table 3: Representative ACCMOTIONDET hyperparameter configurations and their corresponding validation-set performance. Note that the “Recall (%)” column displays the retained recall alongside the absolute recall drop relative to the baseline in parentheses, i.e., Recall (-Recall Drop). The best and second-best results for each metric are highlighted in **bold** and *italic underline*, respectively. Note that Accumulation Step $\omega = 5$ and Decay rate $\delta = 1$ are fixed across all configurations.

Method	Parameters						Metrics				Ranking Score
	Scale factor α	Block size B	Block active threshold τ	Diff threshold τ_d	Activation threshold τ_m	Acc. cap $K_{\max}$	Recall (%) $\uparrow$	Skip Rate (%) $\uparrow$	FAR (%) $\downarrow$	FPS $\uparrow$	
Baseline (No Skip module)	-	-	-	-	-	-	94.347 (-0.000)	0.00	0.740	24.66	-
AccMotionDet	0.5	32	0.05	5	5	15	93.895 (-0.452)	44.39	<u>0.737</u>	36.32	0.8020
AccMotionDet	0.5	32	0.1	5	5	25	93.326 (-1.021)	<u>55.09</u>	<u>0.737</u>	<u>41.15</u>	<u>0.7912</u>
AccMotionDet	0.5	32	0.05	5	5	25	93.923 (-0.425)	42.89	<u>0.737</u>	35.71	0.7903
AccMotionDet	0.5	32	0.1	5	5	35	93.354 (-0.993)	54.42	<u>0.737</u>	40.77	0.7898
AccMotionDet	0.5	32	0.05	5	5	35	93.929 (-0.418)	42.38	<u>0.737</u>	35.52	0.7856
AccMotionDet	0.5	32	0.1	5	5	15	93.217 (-1.131)	56.30	<u>0.737</u>	41.79	0.7785
AccMotionDet	0.5	32	0.1	3	5	15	<u>94.019</u> (-0.329)	39.76	<u>0.737</u>	34.47	0.7756
AccMotionDet	0.5	32	0.1	3	5	25	<u>94.019</u> (-0.329)	39.36	<u>0.737</u>	34.32	0.7705
AccMotionDet	0.5	32	0.1	3	5	35	<u>94.019</u> (-0.329)	39.09	<u>0.737</u>	34.21	0.7671
AccMotionDet	0.5	32	0.05	3	10	15	93.813 (-0.534)	42.63	0.735	35.69	0.7591

accumulated score to arbitrarily large values, such that the subsequent temporal decay requires excessive time to clear the score once motion ceases. By selecting a range of values, we diversify the system’s saturation behavior, allowing us to tune how quickly the module resets following high-activity periods.

Table 3 presents representative configurations of the ACCMOTIONDET-based skip module evaluated on the validation set. The results show a marked improvement over FRAMEDIFFDET, with several configurations achieving skip rates of 39%–56% while maintaining high recall (above 93%, relative to the baseline of 94.347%), remaining within the safety constraint $\delta_R = 0.015$. The best-ranked configuration ($\alpha=0.5$, $B=32$, $\tau=0.05$, $\tau_d=5$, $\tau_m=5$, $K_{\max}=15$) achieves a skip rate of 44.39% with a recall drop of only 0.452%, yielding a throughput of 36.32 FPS — a substantial improvement over the 24.66 FPS baseline.

4.4.2 Frame-Level Performance and Throughput: Baselines and Skip Module

Table 4: Frame-level performance and throughput of the proposed pipeline against lightweight fire-and-smoke detectors on the test set. “Our Big Model (with Skip Module)” refers to the full proposed pipeline. Recall, FAR, Accuracy, and Precision are reported in percentage (%); FPS is averaged across all frames in the test videos. GFLOPs denotes the static per-frame floating-point operations (10^9) of the detection model. The best and second-best values per metric are highlighted in **bold** and *italic underline*, respectively.

Method	Recall (%) $\uparrow$	FAR (%) $\downarrow$	Accuracy (%) $\uparrow$	F1 Score $\uparrow$	Precision (%) $\uparrow$	FPS $\uparrow$	GFLOPs $\downarrow$	Model size (Mb) $\downarrow$
MobileNet [33]	5.840	<u>0.179</u>	77.490	0.110	91.056	35.15	<u>1.14</u>	20.60
FireNet [17]	34.097	13.467	74.074	0.385	44.107	48.12	0.02	7.45
YOLOv5l [10]	49.360	3.609	85.216	0.613	81.000	<u>78.08</u>	107.70	88.50
YOLOv5s [10]	68.584	10.392	84.612	0.679	67.287	109.94	15.80	<u>13.70</u>
Our Big Model (without Skip Module)	95.616	0.251	98.767	0.974	<u>99.166</u>	24.97	30.61	430.00
Our Big Model (with Skip Module)	<u>94.384</u>	0.136	<u>98.562</u>	<u>0.969</u>	99.541	32.32	30.61	430.00

Table 4 benchmarks the proposed pipeline (ACCMOTIONDET skip module with the optimal configuration paired with the

BIG Model) against lightweight fire-and-smoke detectors, described in Section 4.3. The baseline BIG Model (without skip module) achieves the highest recall (95.62%), accuracy (98.77%), and F1-score (0.97), substantially outperforming all lightweight alternatives. This gap reflects the consequences of neural scaling laws [40], [41], [42]: the BIG Model benefits from both a more expressive architecture and a significantly larger training dataset. Notably, MobileNet and FireNet exhibit near-degenerate recall of 5.84% and 34.1%, respectively, rendering them unsuitable for safety-critical fire and smoke detection despite their speed and compactness. YOLOv5s achieves the highest throughput (109.94 FPS) but still suffers a recall of 68.58%, roughly 27 percentage points below the BIG Model, confirming that no lightweight alternative achieves an acceptable accuracy–efficiency balance on this task.

Effect of the Skip Module: Integrating the skip module yields a consistent set of improvements across nearly all metrics. Throughput increases from 24.97 to 32.32 FPS, a gain of approximately 29%, while the F1-score is almost fully preserved at 0.969. Recall drops by only 1.23 percentage points (95.616% $\rightarrow$ 94.384%), remaining within the safety constraint $\delta_R = 0.015$. A less obvious but notable benefit is the reduction in FAR from 0.251% to 0.136%, accompanied by a precision increase from 99.166% to 99.541%: by suppressing inference on near-static background frames, the skip module also eliminates a class of incorrect detections that the BIG Model would otherwise generate.

4.4.3 Comparison with Temporal Post-Processing

Temporal Persistence Thresholding (TPT) [10] augments the standard per-frame inference pipeline with a lightweight post-processing stage designed to suppress isolated false alarms. Unlike the proposed skip module, TPT does not reduce the number of frames forwarded to the classifier — every frame is still processed by the classification model. Instead, a circular boolean buffer of length W records the raw per-frame predictions over a rolling window. A detection is confirmed only if the fraction of positive predictions within the buffer exceeds a persistence threshold τ_{persist} ; otherwise, the prediction is suppressed and the frame is relabelled as negative.

Table 5: Representative hyperparameter grid search results for Temporal Persistence Thresholding (TPT) on the validation set. The best and second-best values per metric are highlighted in **bold** and *italic underline*, respectively.

Experiment	Window Size W	Persist Threshold τ_p	Recall (%) $\uparrow$	FAR (%) $\downarrow$	Accuracy (%) $\uparrow$	F1 Score $\uparrow$	Precision (%) $\uparrow$	FPS $\uparrow$
Big Model without Skip Module (Baseline)	-	-	94.347	0.740	98.260	0.957	97.020	24.66
TPT method (Conservative: safety-first)	5	0.2	<u>94.217</u>	<u>0.739</u>	<u>98.235</u>	<u>0.956</u>	<u>97.023</u>	<u>24.65</u>
TPT method (Balanced between Recall Drop and FAR decrease gain)	10	0.5	93.696	0.732	98.135	0.953	97.034	<u>24.65</u>

Table 6: Frame-level performance comparison between the no-skip baseline, TPT variants, and the proposed pipeline (Big Model with AccMotionDet skip module) on the test set. The best and second-best values per metric are highlighted in **bold** and *italic underline*, respectively.

Method	Recall (%) $\uparrow$	FAR (%) $\downarrow$	Accuracy (%) $\uparrow$	F1 Score $\uparrow$	Precision (%) $\uparrow$	Skip Rate (%) $\uparrow$	FPS $\uparrow$
Baseline (notemp)	95.616	0.251	98.767	0.974	99.166	0.00	24.97
TPT _{strict} ($W = 5$, $\tau_{\text{persist}} = 0.2$)	95.519	0.250	98.745	<u>0.973</u>	99.168	0.00	24.96
TPT _{balanced} ($W = 10$, $\tau_{\text{persist}} = 0.5$)	95.123	<u>0.247</u>	98.653	0.971	<u>99.174</u>	0.00	24.96
Big Model with Skip module (AccMotionDet)	94.384	0.136	98.562	0.969	99.541	34.93	<u>32.32</u>

We follow the hyperparameter selection protocol of [10], evaluating TPT on the validation set $\mathcal{D}_{\text{val}}$. Two representative configurations are reported: a strict variant (TPT_{strict}: $W = 5$, $\tau_{\text{persist}} = 0.2$) and a balanced variant (TPT_{balanced}: $W = 10$, $\tau_{\text{persist}} = 0.5$), with the full grid search results on the validation set shown in Table 5. Both configurations are then compared against the proposed pipeline on test set in Table 6.

Table 6 highlights a clear distinction between temporal post-processing and the proposed skip module in terms of the efficiency–accuracy trade-off. TPT processes every frame through the full Big Model and therefore does not improve throughput, with FPS remaining essentially unchanged (24.97 to 24.96). In contrast, the AccMotionDet-based skip module skips 34.93% of negative frames and increases FPS from 24.97 to 32.32, corresponding to an approximately 29% throughput gain. It also achieves the largest reduction in FAR, decreasing from 0.251% to 0.136% (about 46% relative reduction), whereas the TPT variants provide only negligible FAR improvement. Among the two TPT settings, TPT_{balanced} attains slightly lower recall than TPT_{strict} (95.123% vs. 95.519%) while producing nearly the same FAR (0.247% vs. 0.250%), indicating limited

sensitivity to the persistence threshold. Finally, the recall reduction of AccMotionDet is modest ($95.616\% \rightarrow 94.384\%$) and remains within the safety constraint $\delta_R = 0.015$.

4.4.4 AccMotionDet Skip Module with Eager Mode

Table 7: Representative validation-set grid search results for ACCMOTIONDET (optimal configuration from Section 3.4) augmented with EAGER mode. The “Recall (%)” column reports retained recall with the absolute drop relative to the baseline in parentheses (i.e., Recall – Recall Drop). The best and second-best results per metric are highlighted in **bold** and *italic underline*, respectively.

Method	Parameters			Metrics				Ranking Score
	W_{fire}	W_{clr}	N_{chk}	Recall (%) $\uparrow$	Skip Rate (%) $\uparrow$	FAR (%) $\downarrow$	FPS $\uparrow$	
Baseline (No Skip module)	-	-	-	94.347 (-0.00)	0.00	<u>0.740</u>	24.66	-
AccMotionDet	-	-	-	<u>93.895</u>	44.39	0.737	36.61	-
AccMotionDet (Eager Mode)	1	7	50	94.347 (-0.00)	<u>43.72</u>	<u>0.740</u>	<u>36.42</u>	0.9946
AccMotionDet (Eager Mode)	1	7	30	94.347 (-0.00)	43.35	<u>0.740</u>	36.26	<u>0.9637</u>
AccMotionDet (Eager Mode)	2	3	50	94.347 (-0.00)	43.25	0.737	36.19	0.9553
AccMotionDet (Eager Mode)	2	7	50	94.347 (-0.00)	43.19	<u>0.740</u>	36.18	0.9501
AccMotionDet (Eager Mode)	3	3	50	94.347 (-0.00)	42.72	0.737	35.96	0.9115
AccMotionDet (Eager Mode)	3	7	50	94.347 (-0.00)	42.67	<u>0.740</u>	35.95	0.9067
AccMotionDet (Eager Mode)	1	7	20	94.347 (-0.00)	42.57	<u>0.740</u>	35.93	0.8985
AccMotionDet (Eager Mode)	2	3	30	94.347 (-0.00)	42.54	0.737	35.89	0.8958
AccMotionDet (Eager Mode)	2	7	30	94.347 (-0.00)	42.47	<u>0.740</u>	35.89	0.8905
AccMotionDet (Eager Mode)	3	3	30	94.347 (-0.00)	41.72	0.737	35.56	0.8275
AccMotionDet (Eager Mode)	3	7	30	94.347 (-0.00)	41.65	<u>0.740</u>	35.54	0.8221

As established in Section 4.4.2, the AccMotionDet-based skip module achieved a throughput improvement of nearly 30% in FPS; however, this came at the cost of a 1.23% reduction in recall, which is likely attributable to missed detections in slow-developing or nearly stationary smoke scenarios. To address this limitation, the Eager mode is introduced as a complementary mechanism to augment the skip module and recover recall in such cases. In this section, the Eager mode is evaluated in conjunction with the AccMotionDet-based skip module, with its hyperparameters fixed to the optimal values identified in Section 4.4.1.2 (i.e., $\alpha=0.5, B=32, \tau=0.05, \tau_d=5, \tau_m=5, K_{\max}=15$). A grid search is then conducted over the Eager mode parameters to identify the optimal configuration, as follows:

- **Confirmation window** $W_{fire} \in \{1, 2, 3\}$: defines the number of consecutive positive predictions required to trigger a transition into EAGER mode. A deliberately small range is chosen to ensure the system reacts swiftly to early signs of fire or smoke.
- **Clearance window** $W_{clr} \in \{3, 5, 7\}$: This parameter defines the number of consecutive negative predictions required to exit eager mode and resume normal skip operation. The range is deliberately set larger than that of W_{fire} , reflecting an asymmetric cost structure: a premature exit risks missed detections, whereas a delayed exit only causes the system to run unnecessary inference on a few extra frames — a minor and acceptable efficiency loss.
- **Forced-check interval** $N_{chk} \in \{10, 20, 30, 50\}$: defines the maximum number of consecutive frames that are skipped in NORMAL mode before the system forces an inference call, irrespective of the skip module decision. This serves as a safety net against false negatives — for instance, slowly developing or settled smoke may produce insufficient inter-frame motion to trigger $\mathcal{S}$, causing the scene to be treated as static and inference to be suppressed indefinitely. If the forced inference yields a positive prediction and the W_{fire} criterion is satisfied, the system transitions into EAGER mode. Smaller values increase check frequency at the cost of redundant inference on truly static scenes, whereas larger values reduce this overhead at the expense of a longer worst-case recovery delay. The selected range provides a systematic sweep from aggressive ($N_{chk} = 10$) to infrequent ($N_{chk} = 50$) checking.

Following the hyperparameter selection protocol described in Section 3.4, the optimal EAGER mode configuration is identified on the validation set $\mathcal{D}_{val}$, with results reported in Table 7.

The best-ranked configuration ($W_{fire} = 1, W_{clr} = 7, N_{check} = 50$) is then evaluated on the test set $\mathcal{D}_{test}$ and compared against the skip-only configuration in Table 8.

Table 8 reports the effect of EAGER mode on system performance. EAGER mode successfully recovers recall on slow-developing or settled smoke scenarios, increasing from 94.384% (ACCMOTIONDET only) to 95.565% (ACCMOTIONDET + EAGER), nearly matching the baseline recall of 95.616% (BIG Model without skip module) — a recovery of critical importance in safety-critical applications. However, this improvement comes with two costs: the skip rate slightly decreases from 34.93% to 34.31%, and the false alarm rate (FAR) increases from 0.136% to 0.251% (the implications of this FAR increase are discussed in detail in Section 4.4.5.3).

Table 8: Comparison of Video-level performance metrics, measuring the delay and initial detection speed. The best and second-best results for each metric are highlighted in **bold** and *italic underline*, respectively.

Method	Recall (%) $\uparrow$	FAR (%) $\downarrow$	Accuracy (%) $\uparrow$	F1 Score $\uparrow$	Precision (%) $\uparrow$	Skip Rate (%) $\uparrow$	FPS $\uparrow$
Baseline (notemp)	95.616	0.251	98.767	0.974	99.166	0.00	24.97
TPT _{strict} ($W = 5, \tau_{\text{persist}} = 0.2$)	95.519	0.250	98.745	<u>0.973</u>	99.168	0.00	24.96
TPT _{balanced} ($W = 10, \tau_{\text{persist}} = 0.5$)	95.123	<u>0.247</u>	98.653	0.971	<u>99.174</u>	0.00	24.96
Big Model with Skip module (AccMotionDet)	94.384	0.136	98.562	0.969	99.541	34.93	<u>32.32</u>
Big Model with Skip module (AccMotionDet + Eager $N_{\text{chk}} = 50, W_{\text{clr}} = 7, W_{\text{fire}} = 1$)	<u>95.565</u>	0.251	<u>98.755</u>	<u>0.973</u>	99.166	<u>34.31</u>	32.54

Despite the reduced skip rate, overall throughput marginally increases from 32.32 to 32.54 FPS. This is explained by the reduced computational overhead in EAGER mode: rather than executing the full ACCMOTIONDET pipeline (Algorithm 3), our implementation converts each frame to grayscale, stores it for subsequent use in NORMAL mode, and applies only temporal decay. Consequently, per-frame computation in EAGER mode is lighter than in NORMAL mode (where both the full-step ACCMOTIONDET skip module and BIG Model inference are active), yielding the observed FPS gain despite the lower skip rate.

4.4.5 Quantitative and Qualitative Analysis

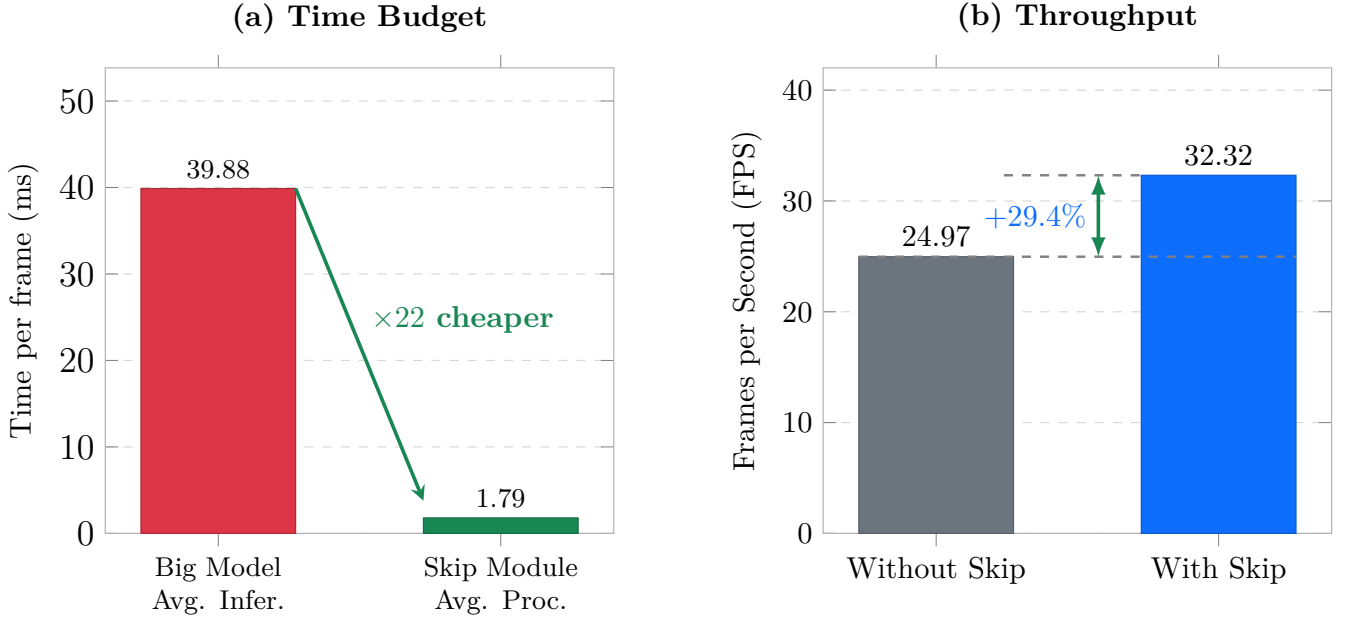


Figure 5: Computational efficiency of the proposed skip-module: (a) average processing time of the skip-module (1.79 ms) compared to the BIG model inference time (39.88 ms), representing a $\times 22$ reduction; (b) system throughput with and without the skip-module, yielding a 29.4% improvement in FPS.

4.4.5.1 Efficiency of Skip Module To evaluate the efficiency of the skip module, the BIG Model was run on the test set $\mathcal{D}_{\text{test}}$ with and without the skip module. The average processing time of the skip module, the BIG Model inference time, and the overall system FPS were recorded in both cases. The results are visualized in Figure 5.

As shown in Figure 5, the skip module requires an average processing time of only 1.79 ms per frame, which is approximately $\times 22$ lower than the BIG Model inference time of 39.88 ms. In the best case, the skip module correctly identifies and bypasses negative frames, contributing only 1.79 ms of overhead. In the worst case, where the skip module fails to bypass a frame and the BIG Model inference is subsequently invoked, the additional latency introduced by the skip module is 1.79 ms — a mere 4.5% increase relative to the BIG Model inference time. Overall, integrating the skip module improves system throughput by approximately 30% on $\mathcal{D}_{\text{test}}$, demonstrating its effectiveness for real-time fire and smoke detection in indoor surveillance

scenarios.

4.4.5.2 Qualitative Analysis To understand how the skip module operates in coordination with the BIG Model, we visualize the inference results of the ACCMOTIONDET skip module on the test set $\mathcal{D}_{\text{test}}$ in RGB images together with the corresponding foreground motion masks generated by the skip module. The qualitative examples are organized into two major categories: success cases and failure cases.

Success Cases

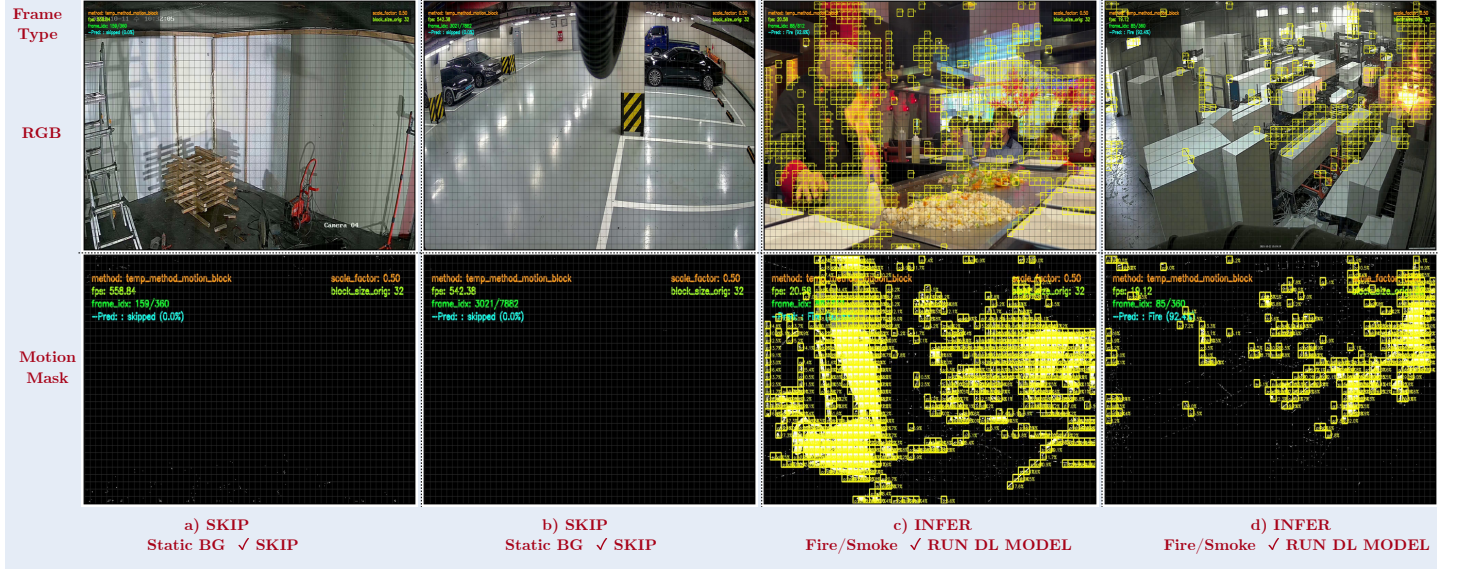


Figure 6: Qualitative results illustrating successful skip module operations. (a) and (b) Static background frames are correctly identified and skipped, avoiding unnecessary inference. (c) and (d) Frames containing active fire or smoke trigger the motion accumulator, correctly prompting the full deep learning model to evaluate the frame.

Figure 6 shows four representative success cases in which the skip module $\mathcal{S}$ operates as intended. These cases can be grouped into two categories:

- **Correctly skipped static background frames** (Fig. 6a,b): ACCMOTIONDET identifies static background frames and skips them without invoking the BIG Model. The corresponding motion masks are empty, indicating no active motion regions.
- **Correctly inferred fire/smoke frames** (Fig. 6c,d): The module correctly detects active fire and smoke regions and forwards these frames for inference. The yellow boxes indicate motion-active blocks detected by ACCMOTIONDET, which are concentrated in the fire/smoke regions, consistent with the continuous nature of these phenomena.

Failure Cases



Figure 7: Qualitative analysis of skip module failure modes. (a) Missed detection: slow-developing or settled smoke fails to trigger the motion accumulator. (b–c) Wasted inferences: inaccurate motion detection OR persistent motion (e.g., human activity) saturates the accumulator, preventing frame skipping. (d–e) Big Model error: frames are correctly passed to the deep learning model, yet the model produces incorrect predictions.

Figure 7 shows five representative failure cases, which can be grouped into three categories:

- **Wrongly skipped frames** (Fig. 7a): Slowly developing smoke or smoke in its settling phase produces motion that is too subtle to trigger the skip module, leading to missed detections.
- **Wrongly passed frames / wasted inference** (Fig. 7b,c): Noise or persistent non-fire motion prevents background frames from being skipped, resulting in unnecessary inference.
- **Incorrect BIG Model predictions** (Fig. 7d,e): Even when fire or smoke frames are correctly forwarded, the BIG Model may still produce an incorrect label, especially when the target occupies only a small portion of the frame.

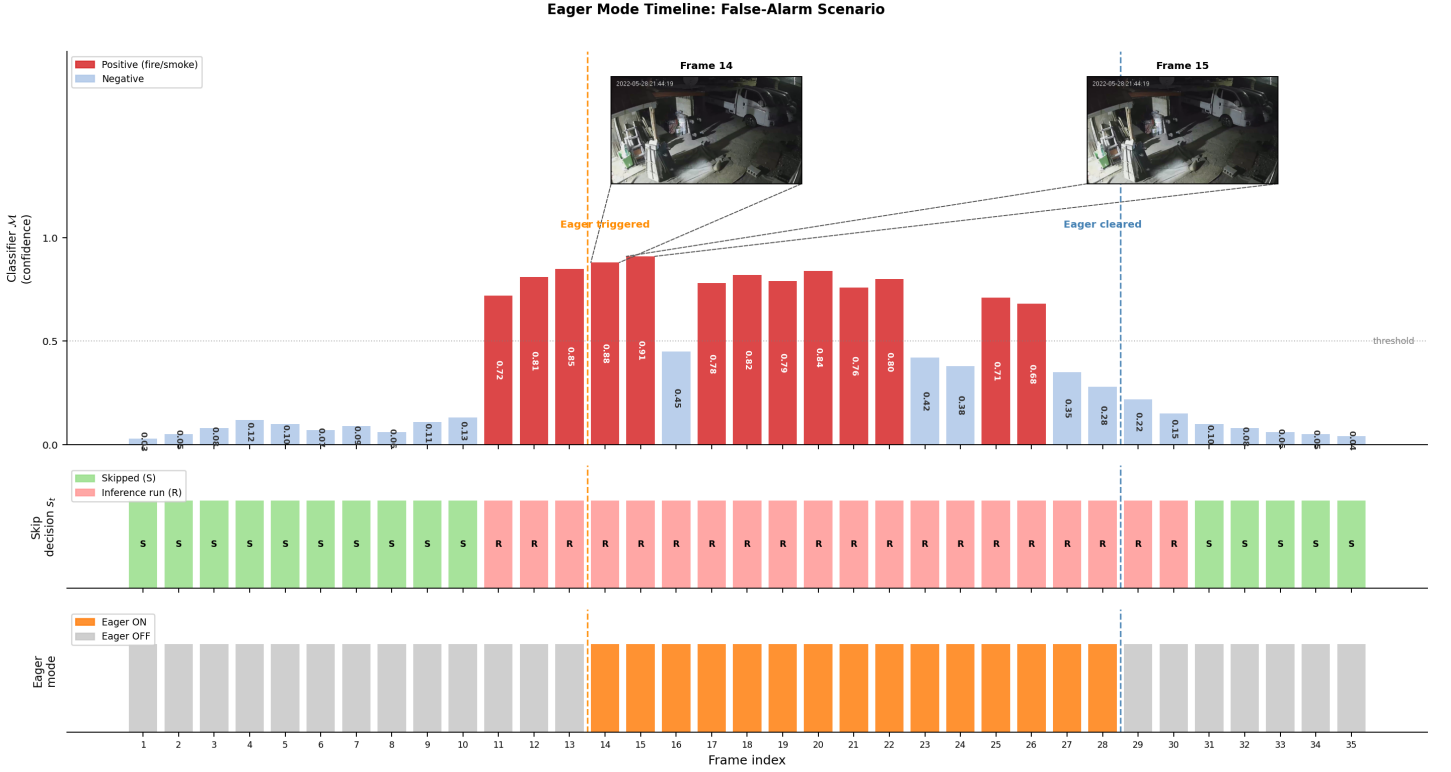


Figure 8: Per-frame timeline illustrating a false-alarm scenario under Eager mode. Once the classifier $\mathcal{M}$ produces W_{fire} consecutive positive predictions on a static ambiguous scene, the system enters Eager mode (orange region), forcing inference on every subsequent frame and causing a sustained increase in FAR until W_{clr} consecutive negative predictions are observed.

4.4.5.3 Analysis of Eager Mode: Recall-FAR Trade-off A notable drawback of Eager mode shown in Table 8 is its tendency to increase false alarms (from 0.136% to 0.251%). Our analysis reveals that Eager mode cannot distinguish false-alarm-prone static scenes from genuine slow-developing or settled smoke events, as both exhibit near-zero inter-frame motion. In such static scenes, the classifier may produce fire/smoke predictions on several consecutive frames, causing the system to enter and remain in Eager mode with prolonged forced inference and elevated FAR. This highlights a fundamental limitation of heuristic, motion-based skip logic: it is insufficient to robustly handle all scenarios. We leave this as an open research question, with potential directions including skip modules that incorporate richer cues (such as color, texture, or learned features) to discriminate between these scene types.

5 Conclusion

We propose a lightweight, plug-and-play skip module $\mathcal{S}$ designed to accelerate real-time fire and smoke detection in indoor surveillance scenarios. Operating at a negligible fraction ($\approx 4.5\%$) of the computational cost of the DL classifier $\mathcal{M}$, the skip module $\mathcal{S}$ efficiently filters out static background frames before they reach $\mathcal{M}$, reducing unnecessary inference overhead. Experiments on a real-world video dataset demonstrate that the proposed approach achieves approximately 30% throughput improvement while sustaining high detection recall, with a recall drop of only 1.23% relative to the baseline. When augmented with EAGER mode, the recall drop is further reduced to 0.05%, nearly fully recovering baseline performance.

Nevertheless, the proposed approach has certain limitations. First, the skip module $\mathcal{S}$ relies solely on inter-frame motion cues, which may be unsuitable for outdoor surveillance scenarios where persistent motion sources — such as swaying vegetation, moving vehicles, etc. — are continuously present, potentially causing $\mathcal{S}$ to pass the majority of frames to $\mathcal{M}$ and negating the efficiency gain. Second, the hyperparameters of $\mathcal{S}$ require dataset-specific tuning prior to deployment, incurring additional optimization overhead. Third, the overall detection accuracy of the system remains bounded by the capacity of $\mathcal{M}$, which is treated as a fixed component and not optimized in this work.

Future work may explore alternative skip strategies — such as leveraging color, texture, or learned features as gating cues — and extend the framework to outdoor surveillance scenarios where motion-based filtering is less effective. Additionally, designing a hyperparameter-free or self-adaptive skip module, as well as jointly optimizing the skip module and the classifier

in an end-to-end training framework, represent promising directions for further improving system efficiency and detection accuracy.

References

- [1] N. F. P. A. (NFPA), “NFPA report - largest fire losses in the united states.” Jan. 1AD. Available: <https://www.nfpa.org/education-and-research/research/nfpa-research/fire-statistical-reports/large-loss-fires-in-the-united-states/largest-fire-losses-in-the-united-states>
- [2] “South korea wildfires become biggest on record as disaster chief points to ‘harsh reality’ of climate crisis | south korea | the guardian.” Jul. 2025. Available: <https://www.theguardian.com/world/2025/mar/27/south-korea-fires-death-toll-rises-worst-in-history>
- [3] G. Cheng, X. Chen, C. Wang, X. Li, B. Xian, and H. Yu, “Visual fire detection using deep learning: A survey,” *Neurocomputing*, vol. 596, p. 127975, 2024.
- [4] D. Gragnaniello, A. Greco, C. Sansone, and B. Vento, “Fire and smoke detection from videos: A literature review under a novel taxonomy,” *Expert Systems with Applications*, vol. 255, p. 124783, 2024.
- [5] X. Geng, Y. Su, X. Cao, H. Li, and L. Liu, “YOLOFM: An improved fire and smoke object detection algorithm based on YOLOv5n,” *Scientific Reports*, vol. 14, no. 1, p. 4543, 2024.
- [6] Z. A. Khan *et al.*, “Optimized cross-module attention network and medium-scale dataset for effective fire detection,” *Pattern Recognition*, vol. 161, p. 111273, 2025.
- [7] R. A. Khan, U. I. Bajwa, R. H. Raza, and M. W. Anwar, “Beyond boundaries: Advancements in fire and smoke detection for indoor and outdoor surveillance feeds,” *Engineering Applications of Artificial Intelligence*, vol. 142, p. 109855, 2025.
- [8] Y. Cao, F. Yang, Q. Tang, and X. Lu, “An attention enhanced bidirectional LSTM for early forest fire smoke recognition,” *IEEE Access*, vol. 7, pp. 154732–154742, 2019.
- [9] M. M. Ali and M. Ghodrati, “Toward reliable fire detection in indoor CCTV footage: Reducing false alarms from benign flames using 3D attention CNNs,” *IEEE Access*, 2025.
- [10] P. V. A. de Venancio, R. J. Campos, T. M. Rezende, A. C. Lisboa, and A. V. Barbosa, “A hybrid method for fire detection based on spatial and temporal patterns,” *Neural Computing and Applications*, vol. 35, no. 13, pp. 9349–9361, 2023.
- [11] D. Gragnaniello, A. Greco, C. Sansone, and B. Vento, “FLAME: Fire detection in videos combining a deep neural network with a model-based motion analysis,” *Neural Computing and Applications*, vol. 37, no. 8, pp. 6181–6197, 2025.
- [12] A. Sobral and A. Vacavant, “A comprehensive review of background subtraction algorithms evaluated with synthetic and real videos,” *Computer Vision and Image Understanding*, vol. 122, pp. 4–21, 2014.
- [13] B. Garcia-Garcia, T. Bouwmans, and A. J. R. Silva, “Background subtraction in real applications: Challenges, current models and future directions,” *Computer Science Review*, vol. 35, p. 100204, 2020.
- [14] Z. Zivkovic and F. Van Der Heijden, “Efficient adaptive density estimation per image pixel for the task of background subtraction,” *Pattern recognition letters*, vol. 27, no. 7, pp. 773–780, 2006.
- [15] A. Ghodrati, B. E. Bejnordi, and A. Habibian, “Frameexit: Conditional early exiting for efficient video recognition,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 15608–15618.
- [16] C. Yu, Z. Mei, and X. Zhang, “A real-time video fire flame and smoke detection algorithm,” *Procedia Engineering*, vol. 62, pp. 891–898, 2013.
- [17] A. Jadon, M. Omama, A. Varshney, M. S. Ansari, and R. Sharma, “FireNet: A specialized lightweight fire & smoke detection model for real-time IoT applications,” *arXiv preprint arXiv:1905.11922*, 2019.
- [18] “Firesense.” Available: <https://www.firesense.eu/>
- [19] M. T. Cazzolato *et al.*, “Fismo: A compilation of datasets from emergency situations for fire and smoke analysis,” in *Brazilian symposium on databases-SBBD*, SBC Uberlândia, Brazil, 2017, pp. 213–223.
- [20] C. R. Steffens, R. N. Rodrigues, and S. S. da Costa Botelho, “An unconstrained dataset for non-stationary video based fire detection,” in *2015 12th latin american robotics symposium and 2015 3rd brazilian symposium on robotics (LARS-SBR)*, IEEE, 2015, pp. 25–30.
- [21] P. V. A. de Venâncio, A. C. Lisboa, and A. V. Barbosa, “An automatic fire detection system based on deep convolutional neural networks for low-power, resource-constrained devices,” *Neural Computing and Applications*, vol. 34, no. 18, pp. 15349–15368, 2022.
- [22] E. Cetin, “Computer vision based fire detection software.” 2015. Available: <http://signal.ee.bilkent.edu.tr/VisiFire/>
- [23] B. C. Ko, S. J. Ham, and J. Y. Nam, “Modeling and formalization of fuzzy finite automata for detection of irregular fire flames,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 21, no. 12, pp. 1903–1912, 2011.
- [24] P. Foggia, A. Saggese, and M. Vento, “Real-time fire detection for video-surveillance applications using a combination of experts based on color, shape, and motion,” *IEEE TRANSACTIONS on circuits and systems for video technology*, vol. 25, no. 9, pp. 1545–1556, 2015.
- [25] G. Lin, Y. Zhang, Q. Zhang, Y. Jia, G. Xu, and J. Wang, “Smoke detection in video sequences based on dynamic texture using volume local binary patterns,” *KSII Trans. Internet Inf. Syst.*, vol. 11, no. 11, pp. 5522–5536, 2017.
- [26] “AI-hub.” Available: <https://aihub.or.kr/aihubdata/data/view.do?dataSetSn=71751>
- [27] L. Huang, G. Liu, Y. Wang, H. Yuan, and T. Chen, “Fire detection in video surveillances using convolutional neural networks and wavelet transform,” *Engineering Applications of Artificial Intelligence*, vol. 110, p. 104737, 2022.

- [28] O. Önal and E. Dandil, “Video dataset for the detection of safe and unsafe behaviours in workplaces,” *Data in Brief*, vol. 56, p. 110791, 2024.
- [29] D. Deniz, E. Ros, E. M. Ortigosa, and F. Barranco, “Optimized edge-cloud system for activity monitoring using knowledge distillation,” *Electronics*, vol. 13, no. 23, p. 4786, 2024.
- [30] M. Rohrbach *et al.*, “Recognizing fine-grained and composite activities using hand-centric features and script data,” *International Journal of Computer Vision*, vol. 119, no. 3, pp. 346–373, 2016.
- [31] R. Marroquin, J. Dubois, and C. Nicolle, “WiseNET: An indoor multi-camera multi-space dataset with contextual information and annotations for people detection and tracking,” *Data in brief*, vol. 27, p. 104654, 2019.
- [32] C. R. Steffens, R. N. Rodrigues, and S. S. da Costa Botelho, “Non-stationary VFD evaluation kit: Dataset and metrics to fuel video-based fire detection development,” in *Latin american robotics symposium*, Springer, 2016, pp. 135–151.
- [33] D. Mukhopadhyay, R. Iyer, S. Kadam, and R. Koli, “FPGA deployable fire detection model for real-time video surveillance systems using convolutional neural networks,” in *2019 global conference for advancement in technology (GCAT)*, IEEE, 2019, pp. 1–7.
- [34] G. Jocher, “YOLOv5 by Ultralytics.” May 2020. doi: [10.5281/zenodo.3908559](https://doi.org/10.5281/zenodo.3908559). Available: <https://github.com/ultralytics/yolov5>
- [35] “Timm/hgnetv2_b5.sslsld_stage2_ft_in1k · hugging face.” Sep. 2025. Available: https://huggingface.co/timm/hgnetv2_b5.sslsld_stage2_ft_in1k
- [36] R. Wightman, “PyTorch image models,” *GitHub repository*. <https://github.com/rwightman/pytorch-image-models>; GitHub, 2019. doi: [10.5281/zenodo.4414861](https://doi.org/10.5281/zenodo.4414861)
- [37] C. Cui *et al.*, “Beyond self-supervision: A simple yet effective network distillation alternative to improve backbones,” *arXiv preprint arXiv:2103.05959*, 2021.
- [38] “PaddleClas/docs/en/models/PP-HGNetV2_en.md at f1233c18455b8acde4fc42ab0bea575fa06daa8e · PaddlePaddle/PaddleClas.” Available: https://github.com/PaddlePaddle/PaddleClas/blob/f1233c18455b8acde4fc42ab0bea575fa06daa8e/docs/en/models/PP-HGNetV2_en.md?plain=1#L33
- [39] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
- [40] J. Hestness *et al.*, “Deep learning scaling is predictable, empirically,” *arXiv preprint arXiv:1712.00409*, 2017.
- [41] I. M. Alabdulmohsin, B. Neyshabur, and X. Zhai, “Revisiting neural scaling laws in language and vision,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 22300–22312, 2022.
- [42] Y. Bahri, E. Dyer, J. Kaplan, J. Lee, and U. Sharma, “Explaining neural scaling laws,” *Proceedings of the National Academy of Sciences*, vol. 121, no. 27, p. e2311878121, 2024.